Unit 1: Introduction to Algorithms (with Diagrams)

1.1 What is an Algorithm?

- An algorithm is a set of instructions for accomplishing a task.
- Algorithms are fundamental to computer science and programming.
- They are not code, but the logic behind the code. You can express an algorithm in plain English, a flowchart, or pseudocode.

Example: A simple algorithm for finding the largest number in a list:

A textual representation of the algorithm:

1. Start.
2. Check if the list is empty. If it is, the process ends.
3. If the list is not empty, set the 'largest_number' to be the first element of the list.
4. Go through the other elements of the list one by one.
5. For each element, check if it is bigger than the 'largest_number'.
6. If it is, update the 'largest_number' to the current element's value.
7. After checking all the elements, return the 'largest_number'.
8. End.

1.2 Binary Search

- Binary search is a classic search algorithm that works on **sorted** lists.
- It follows a "divide and conquer" strategy.
- It's significantly faster than simple search (checking each element one by one).

How Binary Search Works

A textual representation of the algorithm:

1. Start with a sorted list and a target value.
2. If the list is empty, the target is not found.
3. If the list is not empty, find the middle element.
4. If the middle element is the target, the target is found.

5. If the middle element is smaller than the target, repeat the process with the right half of the list.
6. If the middle element is larger than the target, repeat the process with the left half of the list.

Example Trace of Binary Search (Searching for 23 in [5, 12, 15, 23, 30, 45, 50])

A textual representation of the algorithm:

Scenario 1: Searching for 23 in [5, 12, 15, 23, 30, 45, 50]

1. The middle element is 23.
2. 23 is the target, so the element is found.

Scenario 2: Searching for 10 in [5, 12, 15, 23, 30, 45, 50]

1. The middle element is 23. 23 is greater than 10, so we search the left half: [5, 12, 15].
2. The new middle element is 12. 12 is greater than 10, so we search the left half: [5].
3. The new middle element is 5. 5 is less than 10, so we search the right half. The right half is an empty list [ ].
4. The list is empty, so the element is not found.

Simple Search vs. Binary Search

Feature	Simple Search	Binary Search
Requirement	Works on any list	Requires a sorted list
Speed	Linear time - O(n)	Logarithmic time - O(log n)
Efficiency	Less efficient	More efficient

1.3 Big O Notation

- Big O notation is used to describe the performance or complexity of an algorithm.
- It describes the **worst-case scenario** for an algorithm's running time.
- It tells you how fast an algorithm grows relative to the input size.

Understanding Logarithms (log n)

A logarithm is the inverse of an exponent. When we say `log base 2 of 8 is 3`, we are asking "how many times do we need to multiply 2 by itself to get 8?". The answer is 3 (2 * 2 * 2 = 8).

In computer science, we usually deal with `log base 2`, which we write as `log2`. This is because many algorithms, like binary search, work by cutting the problem in half repeatedly.

How to solve logarithms manually (for powers of 2):

If you have a number `n` that is a power of 2 (e.g., 2, 4, 8, 16, 32, ...), you can find `log2(n)` by asking "what power do I need to raise 2 to, to get n?".

- `log2(8)` = 3 (because $2^3 = 8$)
- `log2(16)` = 4 (because $2^4 = 16$)
- `log2(128)` = 7 (because $2^7 = 128$)

For binary search, `log2(n)` tells you the maximum number of of steps it will take to find an element in a sorted list of size `n`.

Common Big O Runtimes Explained

Big O Notation	Name	Explanation	Example
O(1)	Constant	Always takes the same amount of time, no matter the input size.	Getting an item from an array by its index.
O(log n)	Logarithmic	The time it takes grows very slowly as the input size increases.	Binary search.
O(n)	Linear	The time it takes is directly proportional to the input size.	Simple search (checking every item in a list).
O(n log n)	Log-Linear	A common runtime for efficient sorting algorithms.	Quicksort, Mergesort.
O(n^2)	Quadratic	The time it takes grows very quickly as the input size increases.	Selection Sort.
O(n!)	Factorial	The time it takes grows extremely fast. Becomes unusable for even small inputs.	Traveling Salesperson Problem (brute-force).

end of unit